IIIT BHUBANESWAR

Department of Computer Science and Engineering

AccessLab Authentication Studio

Account Registration and Password Verification

Object Oriented Programming Mini Project Report

Submitted by

Subham Mohanty

Roll number b425058

CSE B Semester III

Academic period 2025 to 2029

Supervisor

Mr Srichandan Shobhanayak

Submission date 1 November 2026

Submitted for academic evaluation

Submission Details and Declaration

AccessLab Authentication Studio is submitted for the Object Oriented Programming mini project assessment in the Department of Computer Science and Engineering at IIIT Bhubaneswar.

Submission item	Details
Student	Subham Mohanty
Roll number and section	b425058 CSE B
Semester	III
Academic period	2025 to 2029
Project supervisor	Mr Srichandan Shobhanayak
Submission date	1 November 2026

Student Declaration

I submit this report as a description of the AccessLab Authentication Studio implementation for my C++ Object Oriented Programming mini project. It identifies the external libraries and technical references used and reports recorded test results with their platform limitations. The demonstration accounts and passwords are fictional.

I am responsible for reviewing the report, understanding the submitted source code, and explaining the class design and demonstration during assessment.

Submitted by Subham Mohanty Roll number b425058

Date 1 November 2026

Acknowledgment

I acknowledge Mr Srichandan Shobhanayak as the project supervisor for this submission and the Object Oriented Programming course context at IIIT Bhubaneswar. I also acknowledge the maintainers of Qt and libsodium. Their libraries provide the native interface, JSON support, and cryptographic operations used by the application.

The references and source directory index identify the technical documentation and implementation files that support the explanations in this report.

Abstract

AccessLab Authentication Studio is a native C++17 application that demonstrates how an account system creates users, stores password verifiers, and authenticates login attempts. A website-style Qt Widgets interface pairs registration and sign-in forms with a live process panel. The panel displays the actual salt, stored password hash, and recomputed login hash produced during the operation. An account inspector shows the saved JSON record, while an OOP explorer connects the running application to its C++ class design.

During registration, libsodium generates a fresh 16-byte random salt and derives a 32-byte Argon2id password hash using a fixed supported cost profile. The account file stores identity metadata, salt, hash, algorithm, and cost settings. It does not serialize the entered password or confirmation. Login reuses the stored salt, recomputes the hash, and compares the resulting bytes in constant time. A successful comparison creates an in-memory signed-in session. An incorrect password does not create a session.

The design uses private account fields for encapsulation, abstract hashing and storage interfaces for abstraction, derived implementations for inheritance, and virtual calls through base-class smart pointers for runtime polymorphism. Composition, constructors, destructors, RAII, STL containers, file handling, and exception handling support the application. The recorded verification completed 17 core C++ scenarios and 11 GUI scenarios on macOS using Qt offscreen rendering. Windows and Linux execution remain unverified.

The result is an educational authentication demonstrator that runs locally without a web server. Its visible account store and process trace support explanation in a college practical examination. It does not claim production website security or protection against an attacker who controls the database file.

Keywords C++17 Object Oriented Programming Argon2id Salt Password Hashing Qt Widgets

Contents

Section	Page
1 Introduction	1
2 Requirements and Scope	2
3 System Architecture	3
4 Object Oriented Design	4 to 5
5 Registration and Login Workflows	6 to 7
6 Password Hashing and Persistent Storage	8 to 9
7 Implementation Details	10
8 Graphical Interface Evidence	11 to 12
9 Verification and Results	13 to 14
10 Build and Demonstration Procedure	15
11 Limitations and Conclusion	16
References and Source Index	17

List of Figures

Figure 1 Class responsibilities and ownership 3

Figure 2 Account registration workflow 6

Figure 3 Login verification workflow 7

Figure 4 Registration form and live process panel 11

Figure 5 Successful login and matching hash values 12

1 Introduction

1 1 Project Context

A login form is easy to recognise, but it does not by itself explain how an application handles passwords. Storing the original password in a readable file exposes it directly, while a purely simulated login can hide the mechanism that actually verifies a user. This project makes the verification path visible while keeping the program small enough to explain as an undergraduate C++ OOP assignment.

AccessLab presents website-style account forms inside a native desktop program. Its account service and local JSON file model the password-verification and database parts of a website. The application performs real hashing and persistent account operations, but it sends no network requests and does not implement an internet-facing website.

1 2 Problem Statement

The project asks how a readable C++ class design can create accounts, save salted password verifiers, authenticate later login attempts, and show the process in a GUI without storing the submitted password directly. It also asks how failure paths can preserve previously saved accounts and leave the application usable.

1 3 Objectives

Implement working registration, login, sign-out, and restart persistence.

Generate an independent random salt for every account and a 256-bit Argon2id output.

Show the actual generated verifier, recomputed login hash, and comparison outcome.

Demonstrate the four OOP pillars through hashing and storage interfaces.

Use bounded input, checked file commits, RAII cleanup, and reproducible tests.

1 4 Expected Outcome

A student can create a fictional account, inspect its saved record, fail a login with a wrong password, authenticate with the correct password, and explain the participating C++ objects. The demonstration separates password hashing from recoverable encryption and makes the limitations of a local teaching tool explicit.

2 Requirements and Scope

The application supports one local account database per process and an in-memory signed-in identity. Its dependencies are the C++ standard library, Qt, and libsodium. CMake builds the executable and test programs.

Functional requirement	Implemented behavior
Account creation	Validate name and normalized email, confirm password, reject duplicate email, hash and save.
Authentication	Recompute a candidate hash using the stored salt and fixed supported settings.
Session state	Create a current account ID only after successful verification and clear it on sign-out.
Persistence	Reload saved accounts after restart and retain the earlier file on pre-commit write failure.
Learning interface	Expose actual verifier fields, process stages, and OOP explanations with fictional records.
Input and file validation	Reject invalid fields, duplicate records, malformed JSON, unsupported profiles, and excessive sizes.

2 1 Technical Requirements

Component	Requirement
Language and compiler	C++17 compatible compiler
Build tool	CMake 3.22 or later
GUI and JSON	Qt 6.5 or later with Widgets and Core
Cryptography	libsodium 1.0.18 or later
Automated GUI checks	Qt Test
Hashing memory	64 MiB per Argon2id operation plus application memory

2 2 Deliberate Boundaries

The project does not include an HTTP server, browser integration, HTTPS, secure web cookies, password recovery, MFA, or network synchronization. The record inspector intentionally remains available while signed out for academic explanation. A deployed website would protect its account database and administration features separately.

3 System Architecture

AuthenticationWindow owns the service and handles presentation. AuthenticationService coordinates identity validation, account collections, and session state. It calls hashing and storage through abstract interfaces rather than directly using library or operating-system details.

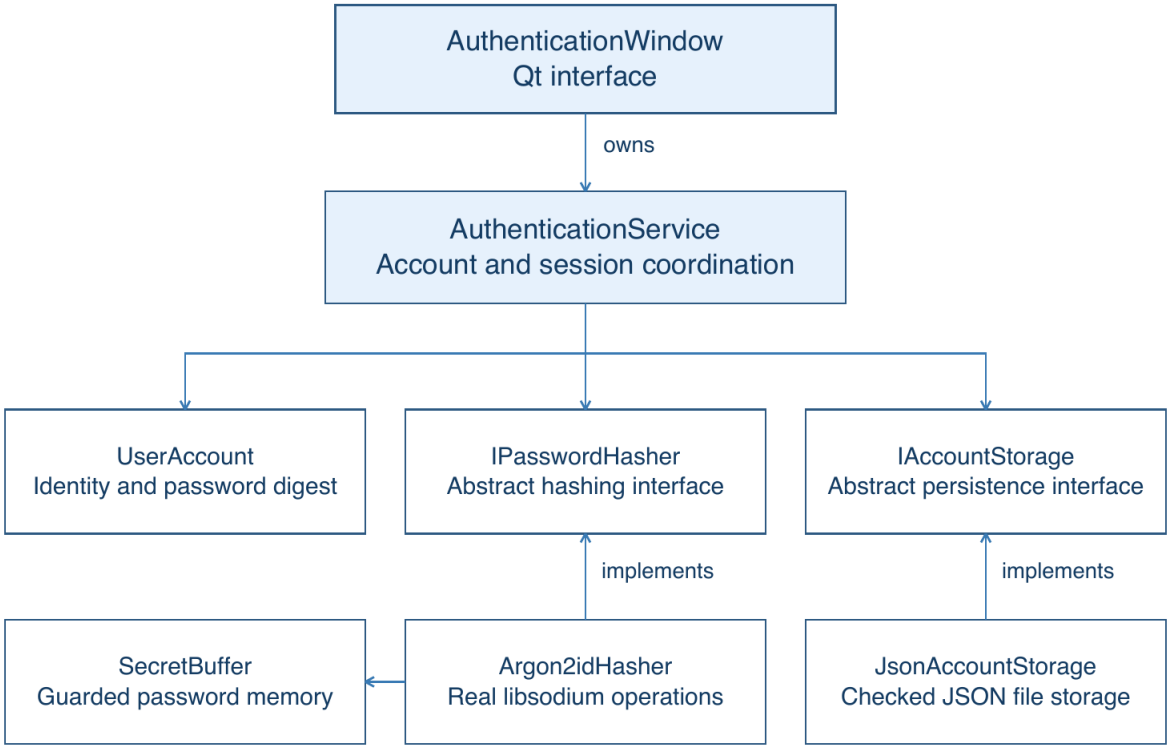


Figure 1 Class responsibilities and ownership

3 1 Responsibilities

Class or component	Responsibility
UserAccount and PasswordDigest	Hold validated identity metadata and fixed-size public salt and hash values.
AuthenticationService	Coordinate registration, login, sessions, and the latest operation trace.
Argon2idHasher	Implement IPasswordHasher using libsodium derivation, randomness, and comparison.
JsonAccountStorage	Implement IAccountStorage with bounded JSON parsing and checked persistence.
SecretBuffer and PlatformFile	Own temporary password memory, file handles, and the cooperative database lock.

4 Object Oriented Design

4 1 Encapsulation

UserAccount keeps its fields private and provides const getters. Its constructor validates identity data. The object stores a PasswordDigest and has no plaintext-password member. Encapsulation controls how existing account data can be accessed and changed.

```
class UserAccount {
    std::uint32_t id_;
    std::string displayName_, email_;
    PasswordDigest digest_;
public:
    const std::string& email() const { return email_; }
    const PasswordDigest& digest() const { return digest_; }
};
```

4 2 Abstraction and Inheritance

IPasswordHasher declares hash and verify operations. IAccountStorage declares load and save. Both have virtual destructors and pure virtual methods. Argon2idHasher and JsonAccountStorage publicly inherit the corresponding interfaces and implement them with override.

```
class IPasswordHasher {
public:
    virtual ~IPasswordHasher() = default;
    virtual PasswordDigest hash(
        const SecretBuffer& password) const = 0;
    virtual VerificationResult verify(
        const SecretBuffer& password,
        const PasswordDigest& stored) const = 0;
};

class Argon2idHasher : public IPasswordHasher {
    // Implements hash and verify with override.
};
```

4 3 Runtime Polymorphism

AuthenticationService holds base-class unique_ptr members. The virtual call hasher_->hash(password) runs Argon2idHasher at runtime, while storage_->save(next) runs JsonAccountStorage. Tests substitute other storage implementations to check error behavior without changing the service logic.

4 Object Oriented Design Continued

4 4 Composition and Resource Ownership

The service has an STL vector of accounts and owns a hasher and storage object. The GUI owns the service. These relationships demonstrate composition. unique_ptr expresses exclusive ownership and releases the corresponding object automatically.

```
std::vector<UserAccount> accounts_;
std::unique_ptr<IPasswordHasher> hasher_;
std::unique_ptr<IAccountStorage> storage_;
```

4 5 Constructors Destructors and RAII

Constructors establish usable state. UserAccount validates its ID, display name, and normalized email. SecretBuffer overloads constructors to allocate a requested byte count or initialize from a string view. Its destructor calls clear, which releases guarded memory through sodium_free. Copy operations are deleted, and move operations transfer ownership. Libsodium attempts to protect and wipe guarded allocations, subject to operating-system limits [2].

```
SecretBuffer::~SecretBuffer() { clear(); }

void SecretBuffer::clear() noexcept {
    sodium_free(bytes_);
    bytes_ = nullptr;
    size_ = 0;
}

SecretBuffer(const SecretBuffer&) = delete;
SecretBuffer& operator=(const SecretBuffer&) = delete;
```

4 6 STL and Exception Handling

vector stores account objects, fixed-size arrays hold salt and hash bytes, and sets detect duplicate IDs or emails. Exceptions stop invalid input, failed derivation, and file errors. The GUI catches failures, clears password fields, displays a message, and keeps the form usable. Qt parent widgets own and delete their child objects [3].

RAII improves cleanup predictability. It does not guarantee that every historical QString, temporary conversion, operating-system buffer, or copied value disappears from memory.

5 Registration Workflow

Registration begins in the GUI. Confirmation must match before the service validates the account and performs expensive hashing. The service proposes an updated collection and publishes it only after storage reports a successful commit.

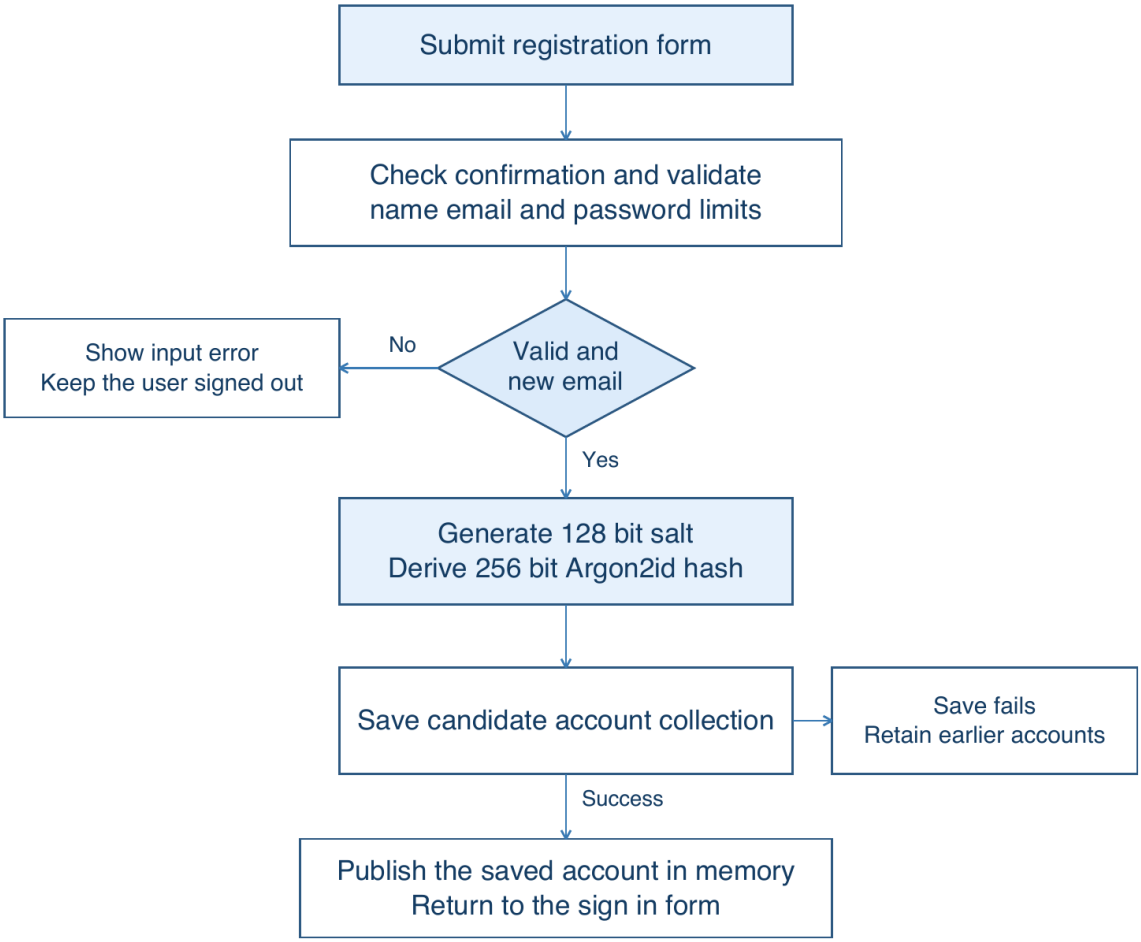


Figure 2 Account registration workflow

A duplicate email or invalid field returns an error without creating an account. A failed write keeps the earlier saved collection and does not publish a new in-memory account. The teaching panel can distinguish a computed verifier from a successfully saved record.

5 Login Workflow

Login normalizes the email, selects a real or dummy verifier, and performs Argon2id using the selected salt. Successful authentication requires both an existing account and matching hash bytes. The session remains local to the running process.

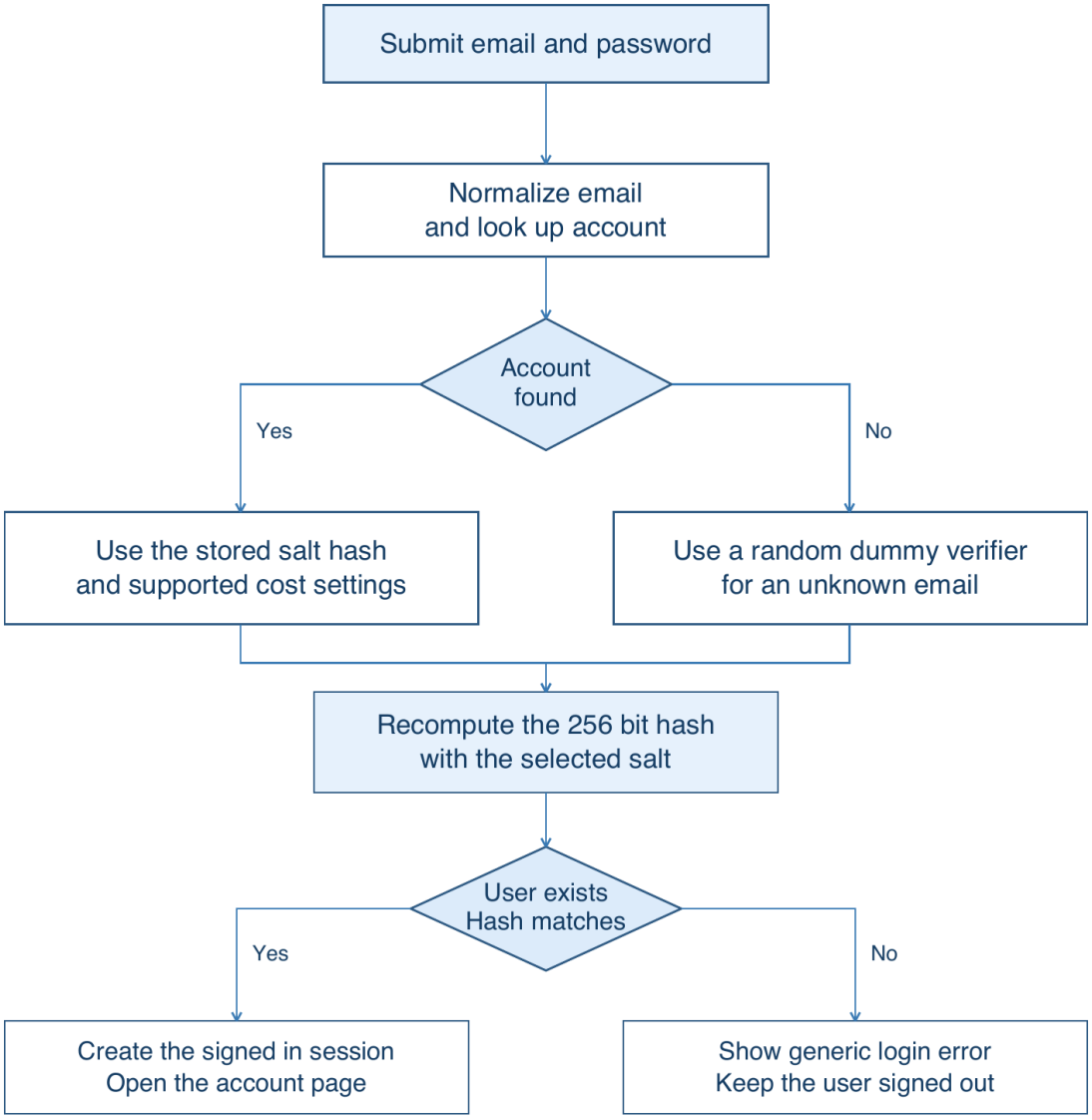


Figure 3 Login verification workflow

An unknown email still receives hashing work with a random dummy record. This reduces a simple difference caused by skipping derivation, but it is not a comprehensive timing or side-channel guarantee. Sign-out clears currentId_. A restart requires authentication again.

6 Password Hashing Parameters

Password verification needs a reproducible verifier rather than a recoverable original password. AccessLab uses libsodium crypto_pwhash with explicit Argon2id 1.3. Its fixed profile accepts the settings below. The library performs the cryptographic algorithm; the application does not implement Argon2id manually [1].

Parameter	Project value	Interpretation
Algorithm	Argon2id 1.3	Memory-hard password derivation
Salt	16 bytes or 128 bits	Fresh random input per account
Hash output	32 bytes or 256 bits	Saved verifier
Operations limit	2	Fixed supported computation setting
Memory limit	67,108,864 bytes	64 MiB per derivation
Hex representation	32 salt characters and 64 hash characters	Each hexadecimal character represents four bits
Comparison	sodium_memcmp	Constant-time equality check for equal-length hashes

6 1 Registration and Login Use Salt Differently

Registration generates an unpredictable salt with randombytes_buf. Login reuses the salt from the account record. A different salt at login would normally derive a different result even from the correct password. The algorithm and cost settings must also remain the same [1].

6 2 Interpretation of Bit Length

The hash output occupies 32 bytes. Multiplying 32 by 8 gives 256 bits. The independent 16-byte salt occupies 128 bits. These sizes describe the representation and derivation inputs, not the guessing strength of a weak user-chosen password.

This profile is a frozen educational baseline rather than a universal deployment recommendation. Increasing a parameter or using the phrase 256 bit does not establish production security. Account database access, password quality, server controls, and session protection would also matter in a deployed system.

6 Persistent Account Storage

JsonAccountStorage stores identity metadata and a password verifier in a documented JSON envelope. The following record came from an executed fictional registration test. It contains the actual saved salt and hash for that test account, not a placeholder cryptographic value.

```
{
  "format": "AccessLabAccounts",
  "version": 1,
  "accounts": [{
    "id": 1,
    "name": "Subham Mohanty",
    "email": "subham@example.test",
    "algorithm": "argon2id-1.3",
    "salt_bits": 128,
    "hash_bits": 256,
    "opslimit": 2,
    "memlimit_bytes": 67108864,
    "salt_hex": "24076c20cbb2519053e5f98257035143",
    "password_hash_hex":
      "5a9ec521b57ef5e5ef942e77e9b2d3cf2b0f18453dc3b19ed61d71b1fb570902"
  }]
}
```

6 3 Validation and Checked Commits

The loader rejects files larger than 2 MiB, unsupported format or cost settings, invalid field sizes, malformed hexadecimal values, duplicate IDs, and duplicate normalized emails. The account limit is 500. It validates the supported profile before login derivation.

Saving writes the serialized candidate collection to an exclusive temporary file in the same directory. The helper checks writes, flush, close, and the platform commit. POSIX uses a no-overwrite link for creation and rename for replacement. Windows uses the corresponding checked native move/replacement operations. A cooperative OS lock limits the database path to one app instance.

The JSON file is intentionally readable and is not authenticated or encrypted as a whole. An attacker with write access can replace verifiers or metadata. The absence of a plaintext password does not make the database tamper-proof.

7 Implementation Details

7 1 Salt Generation and Hashing

The hashing implementation constructs a digest, fills its salt using libsodium randomness, and calls a checked derivation helper. PasswordDigest defines a 16-byte salt and 32-byte hash. These actual source statements show the separation between account logic and cryptography.

```
PasswordDigest Argon2idHasher::hash(
        const SecretBuffer& password) const {
    PasswordDigest digest;
    randombytes_buf(digest.salt.data(), digest.salt.size());
    digest.hash = derive(password, digest.salt);
    return digest;
}
```

7 2 Password Verification

```
VerificationResult Argon2idHasher::verify(
        const SecretBuffer& password,
        const PasswordDigest& stored) const {
    VerificationResult result;
    result.candidate = derive(password, stored.salt);
    result.matched = sodium_memcmp(
        result.candidate.data(), stored.hash.data(),
        stored.hash.size()) == 0;
    return result;
}
```

7 3 Publishing an Account After Persistence

```
const auto digest = hasher_->hash(password);
auto next = accounts_;
next.emplace_back(largest + 1, name, trace_.email, digest);
storage_->save(next);
accounts_.swap(next);
```

The proposed vector stays separate until the commit succeeds. A thrown file exception prevents the final swap. Login stores the active account ID only after a real account exists and the comparison succeeds. It does not rewrite the JSON file or serialize the submitted password or candidate hash.

8 Graphical Interface Evidence

8 1 Account Creation and Process Inspection

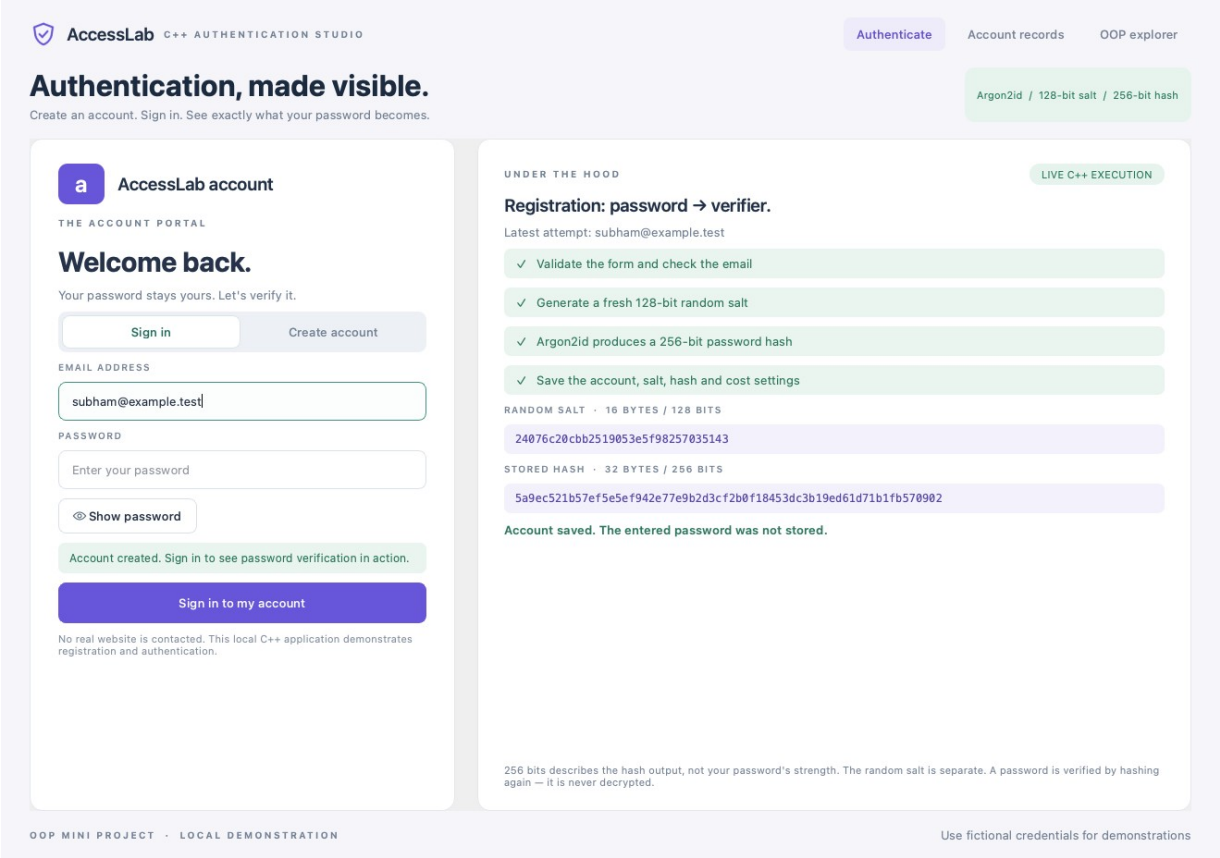


Figure 4 Registration completion and the live process panel

The left side contains website-style account controls. The form initially requests a full name, email address, password, and confirmation. Password input is hidden by default. After a successful registration, the application clears password fields and returns to the sign-in mode with a confirmation message.

The right side shows the completed validation, salt generation, derivation, and save steps. It displays the salt and hash returned by the real cryptographic operation. If persistence fails after hashing, the display identifies a computed hash that was not saved.

Account records opens a read-only table and JSON preview. OOP explorer provides explanations and source snippets for the class design. These inspection views are educational features for fictional data, rather than production website administration controls.

8 Graphical Interface Evidence Continued

8 2 Successful and Rejected Login

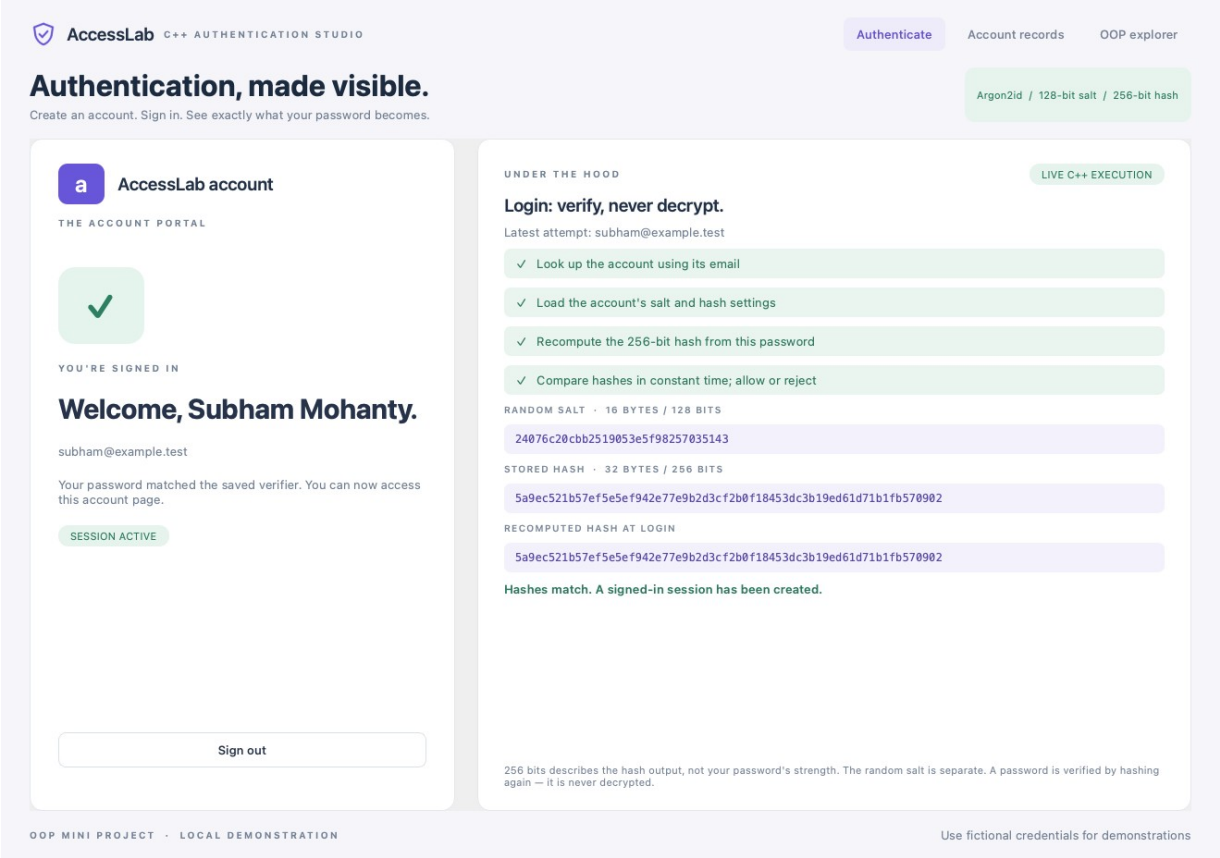


Figure 5 Successful login and matching stored and recomputed hashes

The signed-in account page appears after the stored and recomputed hash bytes match for an existing user. The visible values are generated by the real operation. The session badge describes the in-memory authenticated state; it does not represent an internet session cookie or persistent token.

A wrong password produces a different candidate verifier and leaves the account page closed. The public form displays a generic credential error. An unknown account also remains signed out, and the service performs dummy verification work rather than skipping Argon2id.

The Sign out control clears the active account ID while keeping the saved database available. Closing and reopening the program reloads account records but starts without an authenticated session. The screenshots use fictional credentials and test records.

9 Core Verification Results

The recorded Release verification ran on 24 October 2026 using macOS 27 on Apple Silicon, AppleClang 21.0.0, CMake 4.4.4, Qt 6.10.0, and libsodium 1.0.22. The core suite completed 17 scenarios. The following table summarizes the checks rather than claiming formal security assurance.

ID	Scenario	Result
1	Real Argon2id output sizes and correct or incorrect verification	PASS
2	Same password with independent random salts	PASS
3	Register normalize email sign in and sign out	PASS
4	Wrong password and unknown account cannot open a session	PASS
5	Duplicate email does not replace an existing record	PASS
6	12 byte and 1024 byte password boundary round trips	PASS
7	Malformed or oversized name and email rejection	PASS
8	Persistence and absence of the entered password in saved data	PASS
9	Virtual calls and dummy verification through abstract interfaces	PASS
10	Failed commit preserves saved and in-memory accounts	PASS
11	Active session identity guards	PASS
12	Malformed JSON profile hex and duplicate record rejection	PASS
13	File size bounds and cooperative file lock release	PASS
14	500 account limit and duplicate serialization protection	PASS
15	Noncopyable SecretBuffer and ownership transfer	PASS
16	Unwritable POSIX directory and private file permissions	PASS
17	Symlink database target rejection	PASS

All core checks used actual libsodium hashing and fictional accounts. A CountingHasher in the polymorphism test delegates to real Argon2id. Memory storage and injected failure implementations exist only to reproduce controlled verification cases.

9 GUI Verification Results

The GUI suite drives actual C++ Qt widgets through Qt Test. It completed 11 scenarios using the offscreen platform, including application-window restart and a genuinely unwritable storage directory. The combined recorded result is 28 passed and zero failed across two CTest entries.

ID	Scenario	Result
1	Masked registration form and no prepopulated account file	PASS
2	Mismatched confirmation and malformed email rejection	PASS
3	Actual salt and hash persistence with cleared password input	PASS
4	Same-password accounts have different salts and hashes	PASS
5	Duplicate account rejection through the form	PASS
6	Wrong password gives a failed comparison and no session	PASS
7	Correct password opens the account page and sign-out works	PASS
8	Record inspector shows verifier fields without the entered password	PASS
9	All eight OOP explorer topics expose source snippets	PASS
10	New application window authenticates persisted accounts	PASS
11	Failed write creates no account and keeps the form enabled	PASS

9 1 Interpretation and Limits

The Release build enabled compiler warning checks and recorded no compiler warnings. The screenshots are direct captures of the compiled widgets. Tests observed different salt and hash values and checked for the absence of a chosen fictional password string in saved data. These observations support the implementation checks but do not prove randomness or general cryptographic security.

Windows and Linux execution and native interactive Cocoa operation were not independently verified. Two POSIX-specific core checks and one POSIX-specific GUI check are excluded on Windows. No pass result on those unexecuted platforms is claimed. The project source and verification record identify the exact exercised cases [4].

10 Build and Demonstration Procedure

10 1 Windows Development Setup

Install Visual Studio 2022 Build Tools with Desktop development with C++, CMake, Git, VS Code, Qt, and libsodium. The supplied configuration supports the vcpkg packages libsodium:x64-windows and qtbase:x64-windows. Use matching compiler and architecture kits. The following commands assume vcpkg is already installed at the shown location.

```
$env:VCPKG_ROOT = "$env:USERPROFILE\tools\vcpkg"
& "$env:VCPKG_ROOT\vcpkg.exe" install `
  libsodium:x64-windows qtbase:x64-windows
$toolchain = "$env:VCPKG_ROOT/scripts/buildsystems/vcpkg.cmake"

cmake -S . -B build -G "Visual Studio 17 2022" -A x64 `
  "-DCMAKE_TOOLCHAIN_FILE=$toolchain" `
  -DVCPKG_TARGET_TRIPLET=x64-windows
cmake --build build --config Release --parallel
ctest --test-dir build -C Release --output-on-failure
.\build\Release\accesslab.exe
```

The PowerShell backtick continues a command on the next line. The complete setup and alternative official Qt-kit instructions are provided in README.md and docs/Setup.md.

10 2 Demonstration Sequence

Create a fictional account and point to the actual salt and hash.

Open Account records and identify the saved verifier fields.

Submit an incorrect password and show the rejected attempt.

Sign in correctly and compare the identical stored and candidate hashes.

Create another account with the same password and show its independent salt.

Use OOP explorer to explain virtual calls and resource ownership.

Restart the application and authenticate the saved account again.

10 3 Other Platforms

On macOS or Linux, configure a C++17 Release build with Qt and libsodium installed, build with CMake, and run CTest. The GUI target is accesslab. Detailed platform commands and the default per-user accounts.json locations remain in the supplied setup guide.

11 Limitations and Conclusion

11 1 Security and Deployment Boundary

The account database stores readable identity metadata and password verifiers. It is not encrypted or authenticated as a complete file. A process that can replace database records can replace a verifier or identity. The program therefore does not claim tamper-proof storage. Cooperative file locking and private POSIX permissions address routine local access and concurrency cases rather than a hostile same-account attacker.

The GUI record inspector intentionally remains available while signed out. Qt text fields, conversion temporaries, OS buffers, and earlier copies may retain sensitive input despite the guarded buffer wrapper. A weak password can still be guessed offline against its stored verifier. A 256-bit output alone does not remove those risks.

11 2 Features Outside This Version

A deployed website would need HTTPS, protected server-side storage, secure web-session handling, request throttling, account recovery policy, and other controls. AccessLab does not implement those network components, MFA, password change, account deletion, or persistent session tokens. Future extensions should preserve the interfaces and add focused verification rather than weaken the current separation of responsibilities.

11 3 Conclusion

AccessLab meets the educational goal of connecting working account authentication with fundamental C++ OOP. Registration creates a salted verifier, login recomputes and compares it, saved accounts survive restart, and error paths protect the earlier state during checked persistence failures. The process panel and record inspector expose the mechanism using real values.

The four pillars are visible in private account fields, abstract interfaces, derived implementations, and virtual dispatch. Composition, smart pointers, RAII, STL, file handling, and exceptions make resource and failure behavior explainable. The recorded 28 passing scenarios provide reproducible implementation evidence within the tested macOS environment, while the report states the remaining platform and security limits.

References and Source Index

[1] Libsodium documentation. The pwhash API and password derivation parameters.
https://libsodium.gitbook.io/doc/password_hashing/default_phf

[2] Libsodium documentation. Secure memory and guarded allocations.
https://doc.libsodium.org/memory_management

[3] Qt documentation. Object Trees and Ownership.
https://doc.qt.io/qt-6/objecttrees.html

[4] AccessLab project record. Recorded verification executed on 24 October 2026.
docs/Verification.md and docs/Test_Output.txt

[5] AccessLab project source. Authentication and hashing implementation.
src/AuthenticationService.cpp and src/Argon2idHasher.cpp

[6] AccessLab project source. JSON persistence and checked platform commits.
src/JsonAccountStorage.cpp and src/PlatformFile.cpp

Online references consulted on 1 November 2026. Project facts and screenshots refer to the submitted implementation and its recorded verification.

Source Directory Index

Directory or file	Contents
include/	UserAccount, PasswordDigest, hashing/storage interfaces, service, SecretBuffer, and platform declarations.
src/	Account validation, hashing, service coordination, JSON storage, memory ownership, and platform file helper.
gui/	Native Qt window, form and inspector construction, events, theme, and entry point.
tests/	C++ core verification and C++ Qt GUI verification.
docs/	OOP explanation, workflow, demonstration guide, screenshots, example record, setup, and recorded results.
CMakeLists.txt	Build targets, linked dependencies, and CTest registration.